

NOIP模拟赛

题目名称	组合	原神	公交	树
题目类型	传统型	传统型	传统型	传统型
目录	comb	genshin	bus	tree
可执行文件名	comb	genshin	bus	tree
输入文件名	comb.in	genshin.in	bus.in	tree.in
输出文件名	comb.out	genshin.out	bus.out	tree.out
每个测试点时限	1s	1s	2s	2s
内存限制	128MB	64MB	256MB	1024MB
测试点数目	20	20	20	23
测试点是否等分	是	是	是	否

提交源程序文件名

对于 C++ 语言	comb.cpp	genshin.cpp	bus.cpp	tree.cpp
-----------	----------	-------------	---------	----------

编译选项

对于C++语言	<code>-std=c++14 -O2</code>
对于其他语言	<code>rm -r -f</code>

注意事项（请仔细阅读）

1. 需要在选手目录下对每一题建立与题目名称相同的子文件夹，并将相应程序放入其中。
2. 文件名（程序名和输入输出文件名）必须使用英文小写。
3. C/C++中函数main()的返回值类型必须是int，程序正常结束时的返回值必须是0。
4. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
5. 选手提交的程序源文件必须不大于 100KB。
6. 程序可使用的栈空间内存限制与题目的内存限制一致。
7. 评测环境: Intel® Core™ i9 – 12900HX Processor 3.8GHz。

组合(comb)

时间限制:1s, 空间限制:128MB

题目描述

给定 q 组正整数 $a, b, c (1 \leq a \leq b \leq c \leq n)$, 对下面的式子求值:

$$\sum_{i=a}^b 2^{c-i} i^2 (c-i) \binom{c}{i}$$

其中 $\binom{c}{i}$ 表示组合数 C_c^i .

由于答案可能很大, 你只需要输出答案对 $10^9 + 7$ 取模的结果.

输入格式

从文件 `comb.in` 中读入数据.

第一行输入两个正整数 n, q , 分别表示 a, b, c 的上界以及测试组数.

第 2 至 $q + 1$ 行每行输入三个正整数 a, b, c , 含义见题目描述.

输出格式

输出到文件 `comb.out` 中.

输出 q 行, 每行输出一个询问的答案.

测试样例

样例1输入

```
5 3
2 2 3
4 5 5
3 4 5
```

样例1输出

```
24
160
880
```

样例2/3

见下发文件中的 `comb/ex_comb2.in` 与 `comb/ex_comb2.out` / `comb/ex_comb3.in` 与 `comb/ex_comb3.out`.

该样例满足 $n \leq 2000, q \leq 2000$ 的限制.

样例4/5

见下发文件中的 `comb/ex_comb4.in` 与 `comb/ex_comb4.out` / `comb/ex_comb5.in` 与 `comb/ex_comb5.out`.

该样例满足 $n \leq 2000, q \leq 10^6$ 的限制.

数据范围与约定

对于 40% 的数据, $n \leq 2000, q \leq 2000$.

对于 100% 的数据, $1 \leq n \leq 2000, 1 \leq q \leq 10^6$.

输入输出规模较大, 请选手使用较快的输入输出方式.

原神(genshin)

时间限制:1s, 空间限制:64MB

题目描述

提示:在文字题面后有形式化题意.

旅行者面前有 n 个瓶子自左向右排列, 每个瓶子中水的高度为 a_i . 旅行者想要喝水, 对于一次喝水, 他选择 n 个瓶子中的 k 个瓶子(k 并未给定, $k > 0$), 由于他不喜欢跳来跳去喝水, 因此他会走到所选的最右面一个瓶子, 然后依次喝掉这个瓶子中的水以及它左边的 $k - 1$ 瓶水.

由于喝水的方案太多了, 他只想知道所有喝水方案喝到水量的异或和.

形式化题意:

给定长为 n 的数列 a , 定义全集 $U = \{i \in \mathbb{Z} \mid 1 \leq i \leq n\}$. 求解下面式子:

$$\bigoplus_{S \subseteq U \wedge S \neq \emptyset} \sum_{i=\max\{S\}-|S|+1}^{\max\{S\}} a_i$$

其中 $|A|$ 代表 A 集合的大小, $\max\{A\}$ 表示 A 集合中最大的数, $\oplus$ 表示异或, $\wedge$ 表示 "合取", 可以理解为"且".

输入格式

从文件 `genshin.in` 中读入数据.

第一行输入一个整数 n , 代表数组长度.

第二行输入 n 个正整数, 表示 a 数组.

输出格式

输出到文件 `genshin.out` 中.

一行输出一个整数, 表示答案.

测试样例

样例1输入

```
3
2 3 4
```

样例1输出

```
9
```

样例解释:

选择 1 位置, 喝水量为 2 .

选择 2 位置, 喝水量为 3 .

选择 3 位置, 喝水量为 4 .

选择 1, 2 位置, 喝水量为 5 .

选择 1, 3 位置, 喝水量为 7 (实际喝水位置为 2, 3).

选择 2, 3 位置, 喝水量为 7 .

选择 1, 2, 3 位置, 喝水量为 9 .

上述方案喝水量的异或和为 9 , 即为答案.

样例2/3

见下发文件中的 `genshin/ex_genshin2.in` 与 `genshin/ex_genshin2.out` / `genshin/ex_genshin3.in` 与 `genshin/ex_genshin3.out` .

该样例满足 $n \leq 2000$ 的限制.

样例4/5

见下发文件中的 `genshin/ex_genshin4.in` 与 `genshin/ex_genshin4.out` / `genshin/ex_genshin5.in` 与 `genshin/ex_genshin5.out` .

该样例满足 $n \leq 2 \times 10^5$ 的限制.

数据范围与约定

对于 10% 的数据, $n \leq 20$.

对于 60% 的数据, $n \leq 2000$.

对于 100% 的数据, $1 \leq n \leq 2 \times 10^5, \sum_{i=1}^n a_i < 2^{31}$.

公交(bus)

时间限制:2s, 空间限制256MB

题目描述

提示:在文字题面后有形式化题意.

小 `fsanitize` 家到学校有一个公交车线路, 学校是终点站. 小 `fsanitize` 去上学的方式是: 走一段路之后在公交车站乘坐公交到达学校.

这段路上一共有 n 个车站, 第 i 个车站到**上一站**的距离是 d_i (d_1 的含义则为第一站到家的距离). 不同的车站到终点的价钱并不相同, 具体的, 每个车站价钱有一个参数 m_i , 从第 i 站坐到终点的价格为 $m_i v$, v 由每次询问给定.

小 `fsanitize` 评估一个方案的代价是他走路的距离加上坐公交车的费用. 他想知道最小的代价, 你能帮她解决这个问题吗?

由于小 `fsanitize` 拥有修改内存的能力, 在一些时候某个 d_i 可能改变.

由于市场波动, 在一些时候某个 m_i 可能改变.

形式化题意:

给定两个长为 n 的序列 d_i, m_i , 每次询问给定一个整数 v , 你需要计算:

$$\min_{i=1}^n ((\sum_{j=1}^i d_j) + m_i v)$$

有 q 组修改, 你需要支持下面两种操作:

1 p f v 修改 d_p 为 f .

2 p f v 修改 m_p 为 f .

在每次修改后你都需要输出答案, 输入的 v 则为询问公式的参数.

输入格式

从文件 `bus.in` 中读入数据.

第一行输入两个正整数 n, q , 分别表示车站个数以及修改次数.

第二行输入 n 个正整数, 表示初始的 d_i .

第二行输入 n 个正整数, 表示初始的 m_i .

接下来的 q 行, 每行输入四个正整数, 含义见形式化题意.

输出格式

输出到文件 `bus.out` 中.

输出 q 行, 每行一个整数, 表示答案.

测试样例

样例1输入

```
5 3
5 3 2 5 3
2 2 4 2 5
2 2 4 1
2 4 5 1
1 1 2 2
```

样例1输出

```
7
7
6
```

样例解释

对于第 1 组询问, d 为 $\{5, 3, 2, 5, 3\}$, m 为 $\{2, 4, 4, 2, 5\}$, 询问参数 v 为 1.

在第 1 个位置上车的代价为: $5 + 1 \times 2 = 7$.

在第 2 个位置上车的代价为: $5 + 3 + 1 \times 4 = 12$.

在第 3 个位置上车的代价为: $5 + 3 + 2 + 1 \times 4 = 14$.

在第 4 个位置上车的代价为: $5 + 3 + 2 + 5 + 1 \times 2 = 17$.

在第 5 个位置上车的代价为: $5 + 3 + 2 + 5 + 3 + 1 \times 5 = 23$.

那么最小的代价为 7.

样例2

见下发文件中的 `bus/ex_bus2.in` 与 `bus/ex_bus2.out`.

该样例满足 $n, q \leq 4000$ 的限制.

样例3

见下发文件中的 `bus/ex_bus3.in` 与 `bus/ex_bus3.out`.

该样例满足 所有询问的 v 相同 的限制.

样例4

见下发文件中的 `bus/ex_bus4.in` 与 `bus/ex_bus4.out`.

该样例满足 $n, q \leq 10^5$ 的限制.

数据范围与约定

对于 40% 的数据, $n, q \leq 4000$.

对于另外 20% 的数据, 所有询问的 v 相同.

对于 100% 的数据, $1 \leq n, q \leq 10^5, 1 \leq p_i \leq n, 1 \leq d_i, m_i, f_i, v_i \leq 10^9$.

树(tree)

时间限制:2s, 空间限制1024MB

题目描述

有一天, 小Y在做作业, 他看到了这样一个问题:

对于一颗满二叉树, 每个节点 u 上有一个非负**实数**权值 w_u , 我们定义该树上所有节点构成的集合为 U , 根节点为 1, 记节点 u 的父亲为 $fa_u = \lfloor \frac{u}{2} \rfloor$, 定义这棵树的权值为: $\sum_{u \in U/\{1\}} (w_u - w_{fa_u})^2$.

现在所有叶子节点的权值 p_i 均给定, 其中 p_i 表示标号从小到大第 i 个叶子结点的点权. 请你指定所有非叶节点的点权, 使得这颗二叉树的权值最小. 你只需要输出最小权值即可, 答案对 $10^9 + 7$ 取模.

这个问题太简单了, 被小Y一眼秒了. 于是他向你提出了下面的问题.

上面的问题会给你个数列 p_i , 我们记这个问题的答案为 $f(p)$.

现在给你一个长为 n 的数列 a_i , 有下面两种操作:

1 1 r v 将区间 $[l, r]$ 中的元素增加 v .

2 1 r 查询 $f(a_{[l..r]})$. 其中 $a_{[l..r]}$ 表示 a 在 $[l, r]$ 中的元素组成的新序列.

本题强制在线.

输入格式

从文件 **tree.in** 中读入数据.

第一行输入两个整数 n, q , 分别表示 a 的长度和操作个数.

第二行输入 n 个非负整数, 表示 a_i .

第 3 至 $q + 2$ 行每行输入三个非负整数 opt'_i, l'_i, r'_i , 则有

$opt_i = opt'_i \oplus lans, l_i = l'_i \oplus lans, r_i = r'_i \oplus lans$, 其中 $lans$ 是上一次 **2** 操作的答案, $\oplus$ 是异或运算.

由于每行只输入三个整数, **1** 操作的 v_i 值由以下代码生成:

```
int _lst = 0;
int get(){
    return _lst = ((_lst + 212001290) ^ 323920323) % 200 + 1;
}
```

选手需要在每次 **1** 操作后调用 **get()** 来获取本次操作的 v_i .

输出格式

输出到文件 **tree.out** 中.

对于每一个 **2** 操作, 输出一行一个整数表示答案. 答案对 $10^9 + 7$ 取模.

测试样例

样例1输入

```
5 2
3 0 4 4 4
2 1 2
500000010 500000010 500000013
```

解密后的输入是:

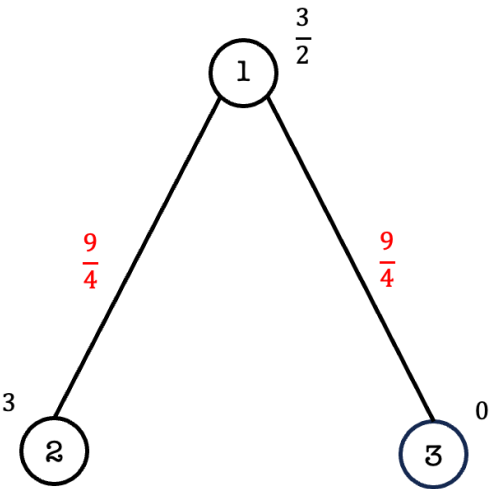
```
5 4
3 0 4 4 4
2 1 2
2 2 5
```

样例1输出

```
500000008
333333345
```

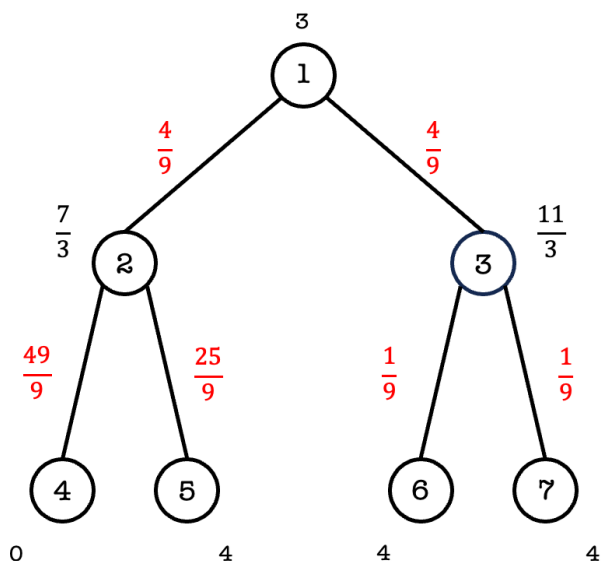
样例解释

对于第一组数据, 查询的 $p_i = \{3, 0\}$, 叶子节点有两个, 一种最优构造如图 (圆圈内是点编号, 外面的黑色字是点权, 红色字是边权):



答案是 $(\frac{3}{2} - 0)^2 + (\frac{3}{2} - 3)^2 = \frac{9}{2}$.

对于第二组询问, 查询的 $p_i = \{0, 4, 4, 4\}$, 叶子节点有四个, 一种最优构造如图:



答案是 $(3 - \frac{7}{3})^2 + (3 - \frac{11}{3})^2 + (\frac{7}{3} - 0)^2 + (\frac{7}{3} - 4)^2 + (\frac{11}{3} - 4)^2 + (\frac{11}{3} - 4)^2 = \frac{28}{3}$.

样例2

见下发文件中的 `tree/ex_tree2.in` 与 `tree/ex_tree2.out` .

该样例满足子任务 1\$ 的限制.

样例3

见下发文件中的 `tree/ex_tree3.in` 与 `tree/ex_tree3.out` .

该样例满足子任务 2 的限制.

样例4

见下发文件中的 `tree/ex_tree4.in` 与 `tree/ex_tree4.out` .

该样例满足 $n \leq 1000$ 的限制.

样例5

见下发文件中的 `tree/ex_tree5.in` 与 `tree/ex_tree5.out` .

该样例满足子任务 3 的限制.

样例6

见下发文件中的 `tree/ex_tree6.in` 与 `tree/ex_tree6.out` .

该样例满足子任务 4 的限制.

样例7

见下发文件中的 `tree/ex_tree7.in` 与 `tree/ex_tree7.out` .

该样例满足子任务 5 的限制.

数据范围与约定

子任务编号	$n, q \leq$	特殊性质
1(10pts)	7	
2(10pts)	15	
3(10pts)		AB
4(20pts)		A
5(30pts)	10^5	
6(20pts)		

特殊性质 A : 对于所有的 1 操作, $l_i = r_i$.

特殊性质 B : 对于所有的 2 操作, $l_i = 1$.

留空则表示无特殊约束.

对于 100% 的数据, $1 \leq n, q \leq 2 \times 10^5, 0 \leq a_i \leq 200, 0 \leq v_i \leq 200, 1 \leq p_i \leq n, 1 \leq l_i \leq r_i \leq n$; 对于每一个 2 操作, 令 $r_i - l_i + 1 = 2^{k_i}$, 则有 $k_i \in \mathbb{Z}, k_i > 0$.

输入输出规模较大, 请选手使用较快的输入输出方式. 请选手注意常数.

我们在下发文件 `tree/sample.cpp` 中提供了快读快写模板以及 `modint` 模板供选手使用, 并提供了 `tree/sample.in` 与 `tree/sample.out` 作为文件输入输出的示例.

我们在下发文件 `tree/tree.cpp` 中提供了一个 `get()` 的使用示例供选手参考.